Challenge 1:

Karim Negm - 11063520

Table of Contents:

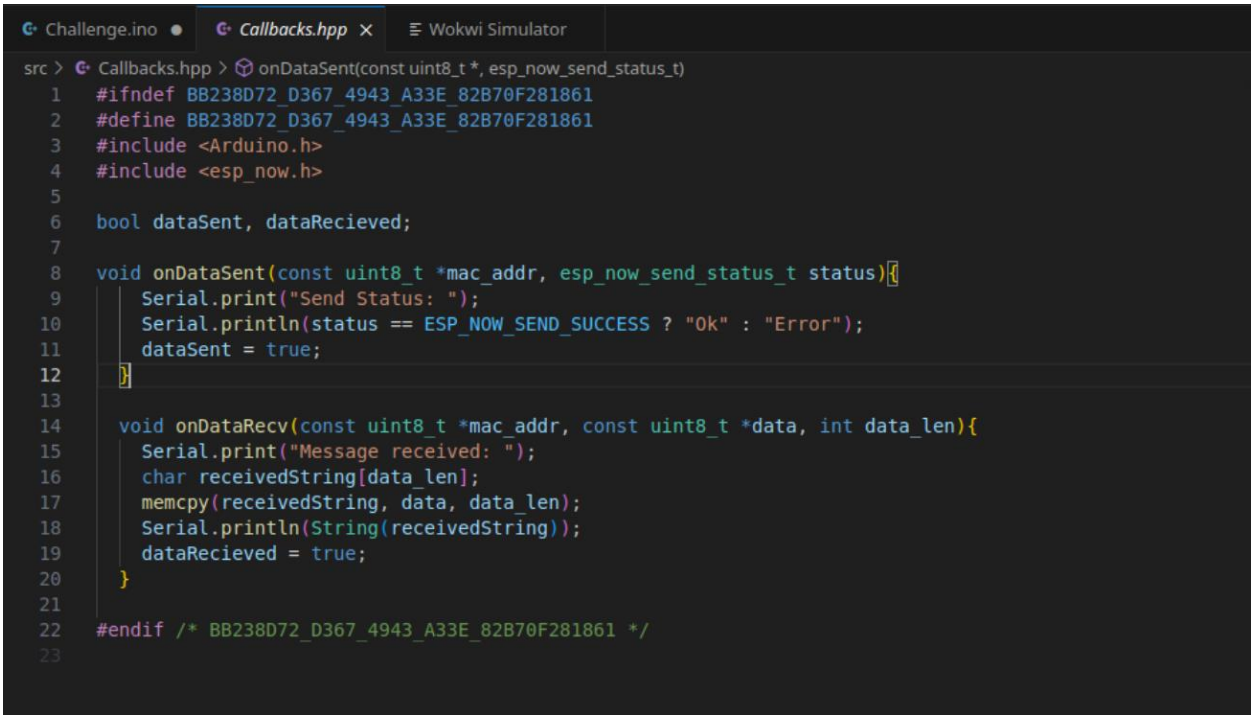
- Project Overview
- Code Explanation
- Power Analysis
- Energy Analysis [Q2]
- Plots and Data Visualizations
- Potential Optimizations [Q3]
- Optimizing Sink Position in a Wireless Sensor Network [Q4]

Overview:

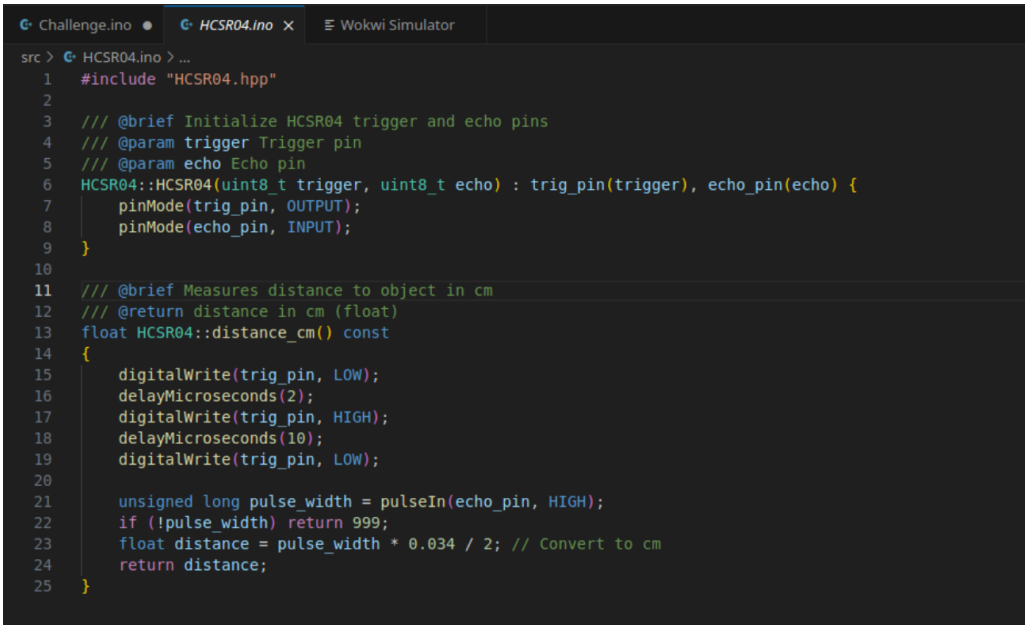
- Objective: Detect parking slot occupancy using an HC-SR04 ultrasonic sensor where distance ≤ 50 cm → occupied.
- Communication: Transmit "FREE"/"OCCUPIED" status to a central ESP32 sink node via ESP-NOW protocol.
- Power Management:
    - Implement deep sleep mode with a duty cycle (wake-up interval X).
    - X is calculated as: Last two digits of team leader's person code (20 from 11063520)
    - X = 20 % 50 + 5 → X= 25 seconds.
    - Node sleeps immediately after transmitting data.

Code Explanation

- Callbacks.hpp
    - Defines the onDataSent callback which is called when the data is transmitted, here we define the Boolean flag “dataSent” we use this to immediately end transmission and deep sleep once data is transmitted.
    - onDataRecv just relays the message to serial when data is received.



- HCSR04 Class
    - Sets up pins, standard function for Ultrasonic distance measurement in cm



- Main code

```
// MAC receiver
const uint8_t broadcastAddress[] = {0x8C, 0xAA, 0xB5, 0x84, 0xFB, 0x90};
esp_now_peer_info peerInfo;
const HCSR04 ultrasonicSensor(15,12);
const float e_booting = TIME_BOOTING * POWER_BOOTING * 1e-3;

void setup() {
  unsigned long start_time = micros();
  // Boot, setup (Idling)
  dataSent = false;
  Serial.begin(115200);
  esp_sleep_enable_timer_wakeup(TIME_TO_SLEEP * uS_TO_S_FACTOR);
  unsigned long idle_time = micros() - start_time ; // Idling Time

  // Scanning
  unsigned long scanning_start_time = millis();
  float dist = ultrasonicSensor.distance_cm();
  String rcv_msg = (dist > 50.0) ? "FREE" : "OCCUPIED";
  unsigned long scanning_time = millis() - scanning_start_time;

  // WiFi On
  unsigned long wifi_start_time = millis();
  WiFi.mode(WIFI_STA);
  esp_now_init();
  esp_now_register_send_cb(onDataSent);
  esp_now_register_recv_cb(onDataRecv);
  memcpy(peerInfo.peer_addr, broadcastAddress, 6);
  peerInfo.channel = 0;
  peerInfo.encrypt = false;
  esp_now_add_peer(&peerInfo);
  unsigned long wifi_time = millis() - wifi_start_time;

  // Transmission
  unsigned long tx_start_time = micros();
  esp_now_send(broadcastAddress, (uint8_t*)rcv_msg.c_str(), rcv_msg.length() + 1);
  while (!dataSent) yield();
  unsigned long tx_time = micros() - tx_start_time;
```

- Idling time is calculated as time from code execution until measurement starts.
    - Includes time setting up serial and enabling sleep timer.
- Measurement time is calculated immediately after. If distance is greater than 50, parking is Free, else, Occupied.
- Standard Wi-Fi setup, esp-now initialized, and callbacks are registered. Wi-Fi time measured from turning on Wi-Fi till beginning transmission.
- Begin transmission and wait for it to finish using the flag defined in the callback.
    - Yield allows background tasks like interrupts, callbacks, and networking to run, so our while loop is no longer blocking.

```
57
58  // Energy calculations, that could be removed in production because Serial takes energy
59  Serial.println("Measured distance (cm): " + String(dist));
60
61  float e_idle = POWER_IDLE * idle_time * 1e-6; // Millijoules
62  Serial.println("Time booting + idle time (ms): " + String(TIME_BOOTING + idle_time * 1e-3));
63  Serial.println("Energy spent (mJ): " + String(e_booting+e_idle));
64
65  float e_scanning = POWER_SCANNING * scanning_time * 1e-3;  // Millijoules
66  Serial.println("Measurement time (ms): " + String(scanning_time));
67  Serial.println("Energy spent measuring (mJ): " + String(e_scanning));
68
69  float e_wifi = POWER_WIFI_ON * wifi_time * 1e-3;  // Millijoules
70  Serial.println("WiFi ON time (ms): " + String(wifi_time));
71  Serial.println("Energy spent with WiFi ON (mJ): " + String(e_wifi));
72
73  float e_tx = POWER_TX_19_5DB * tx_time * 1e-6;  // Millijoules
74  Serial.println("Transmission time (ms): " + String(tx_time * 1e-3));
75  Serial.println("Energy spent transmitting (mJ): " + String(e_tx));
76
77  float e_active = e_booting + e_idle + e_scanning + e_wifi + e_tx;
78  float e_total = e_active + POWER_SLEEPING * TIME_TO_SLEEP;
79  Serial.println("Total energy for active cycle (mJ) =  " + String(e_active));
80  Serial.println("Total energy for complete cycle (mJ) =  " + String(e_total));
81
82  float cycle_time = (TIME_BOOTING + idle_time * 1e-3 + scanning_time + wifi_time + tx_time * 1e-3)*1e-3 + TIME_TO_SLEEP;
83  float total_cycles = BATTERY_ENERGY/(e_total*1e-3);
84  float total_time = total_cycles * cycle_time / 3600; // hours
85  Serial.println("Estimated battery life = " + String(total_time) + " hours");
86
87
88  Serial.flush();
89  esp_deep_sleep_start();
90 }
91
92 void loop() {}
```

- And finally, we use the times we measured to estimate energy consumption, this is discussed in detail in the next section of the report.
- Log the data to serial, then immediately deep sleep, this section of the code is not timed or measured in how much energy it would consume as it is meant to be removed before deployment.

Power Consumption Analysis:

We begin by finding the mean power draw associated with each state, given the CSV files, we can plot the data and estimate the power ranges for each state visually, then compute the mean.

Using a MATLAB script to load the CSV files and find the mean power draws, we find the following.

Power Consumption in Different States:

- Sleeping: 59.6214 mW
- Booting: 371.3775 mW
- Idle: 331.5758 mW
- Scanning: 466.7448 mW
- Wi-Fi On: 704.1771 mW
- TX 2dB: 786.61 mW
- TX 19.5dB: 1221.76 mW

MATLAB Script:

```
sensor_read = readtable("sensor_read.csv");
deep_sleep = readtable("deep_sleep.csv");
transmission_power = readtable("transmission_power.csv");

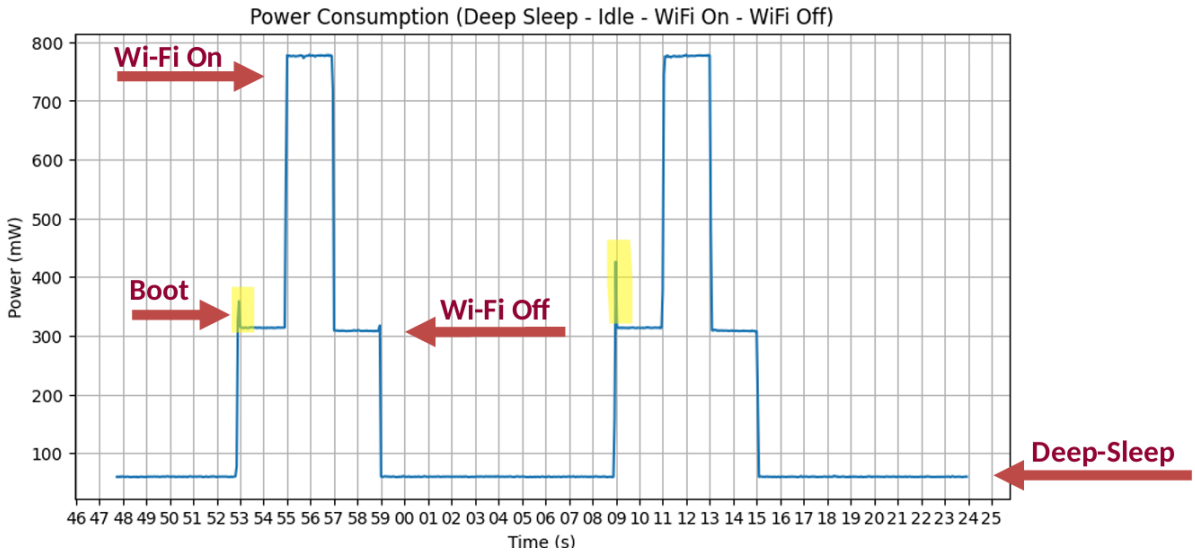
% Threshold ranges estimated visually from plots
booting = isbetween(deep_sleep.Data,320,450);
sleeping = deep_sleep.Data <= 61;
idle = sensor_read.Data < 333;
scanning = sensor_read.Data > 450;
wifiOn = transmission_power.Data < 710;
tx_2db = isbetween(transmission_power.Data,710,810);
tx_195db = transmission_power.Data > 1000;

% Compute mean power consumption for different states
P_booting = mean(deep_sleep.Data(booting));
P_sleeping = mean(deep_sleep.Data(sleeping));
P_idle = mean(sensor_read.Data(idle));
P_scanning = mean(sensor_read.Data(scanning));
P_wifiOn = mean(transmission_power.Data(wifiOn));
P_tx_2db = mean(transmission_power.Data(tx_2db));
P_tx_19_5db = mean(transmission_power.Data(tx_195db));
```

Notice that, due to conflicting data from each CSV file, decisions had to be made from which files the power draw for each state taken. In this project I chose:

- Booting and sleeping powers from “deep_sleep.csv”
- Idling and scanning powers from “sensor_read.csv”
- Wi-Fi and transmissions states from the “transmission_power.csv”

Booting state refers to the power spike upon waking up from deep sleep, seen in this plot, different from idling where the ESP-32 is in a steady-state not doing much.



From this plot we also estimate the time it takes to boot the ESP-32, I believe this is essential for a more accurate energy analysis, as boot times cannot be measured accurately in emulation, it's from this plot estimated that the boot time should be somewhere between 50 and 100ms, difficult to tell accurately given the 20hz sample rate, so it's assumed as 80 milliseconds, which is in-line with values I've found online for the ESP-32 where ~100ms is typical.

I believe it's essential to consider startup time, as booting ends up being one of the largest power consumers in the active state, as we'll see later in this report.

Energy Analysis [Q2]

Now that we have the power data, to find the energy we need the time spent in each state, running the emulation on Wokwi, we get the following times spent in each state:

Empirical data from the ESP32 emulation

- Boot time = 0.08 % seconds
- Idle time = 0.001 % seconds
- Measure time = 0.025 % seconds
- Wi-Fi time = 0.192 % seconds
- Tx time = 0.00019 % seconds [~0.19 ms]
- Sleep time = 25 % seconds

Again, using a MATLAB script, we estimate energy consumption, giving us the following results:

- Total cycle time: 25.298 seconds
- Active time: 0.298 seconds
- Duty cycle: 1.18%

- Average power consumption: 65.92 mW
- **Total energy per cycle: 1.668 J**
- **Time the sensor node lasts before changing the battery: 78.04**
    - Y = 11063520 → 3520 % 5000 + 15000 = 18520 J
    - Number of Cycles = Battery Energy / Energy per Cycle
    - Number of Cycles = 18520 / 1.668 = 11103.1 cycles
    - Time the sensor node lasts before changing the battery is Cycle Time * Number of Cycles →
      25.298s * 11103.1 = 280886.2 seconds = 78.04 hours

MATLAB Script:

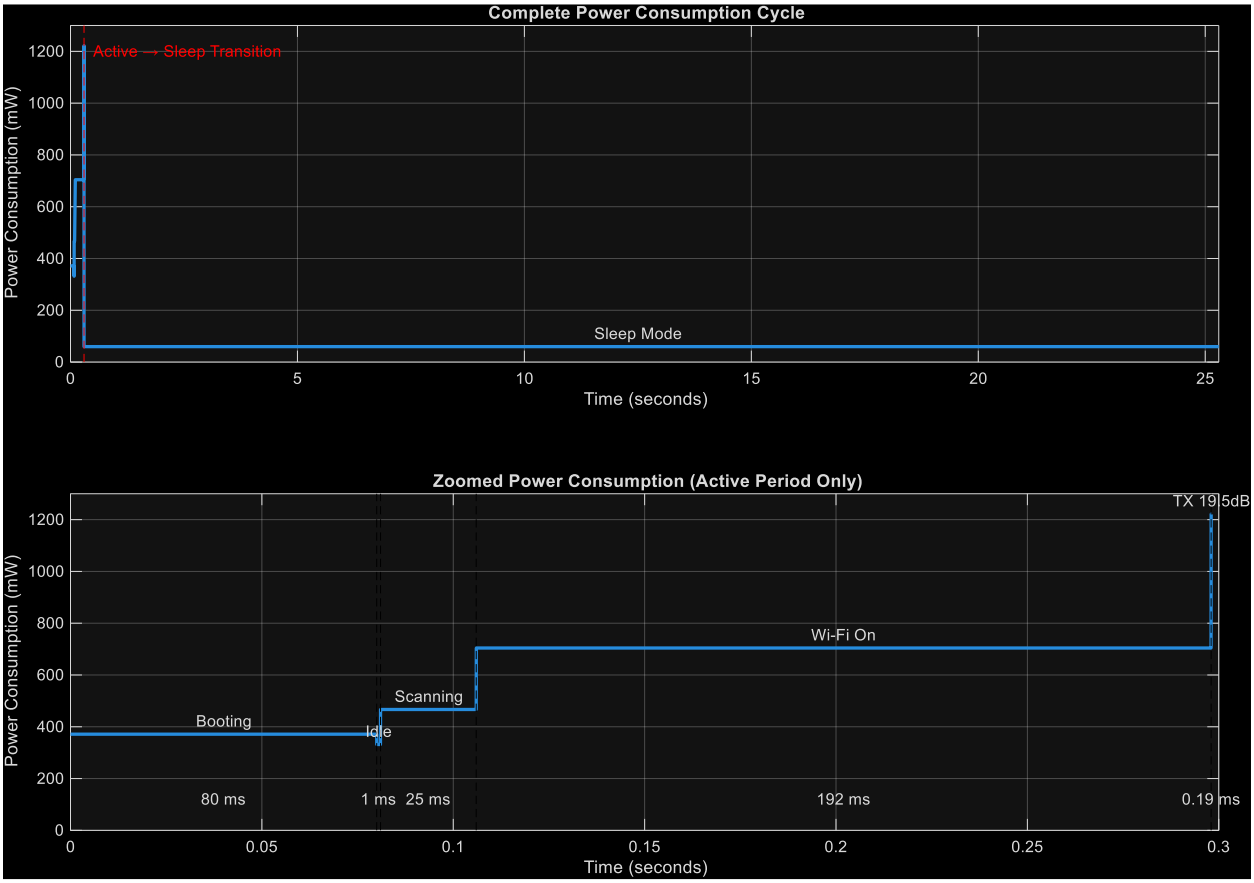
```
% Energy consumption data
boot_energy = P_booting/1000 * boot_time;
idle_energy = P_idle/1000 * idle_time;
measure_energy = P_scanning/1000 * measure_time;
wifi_energy = P_wifiOn/1000 * wifi_time;
tx_energy = P_tx_19_5db/1000 * tx_time;
sleep_energy = P_sleeping/1000 * sleep_time;

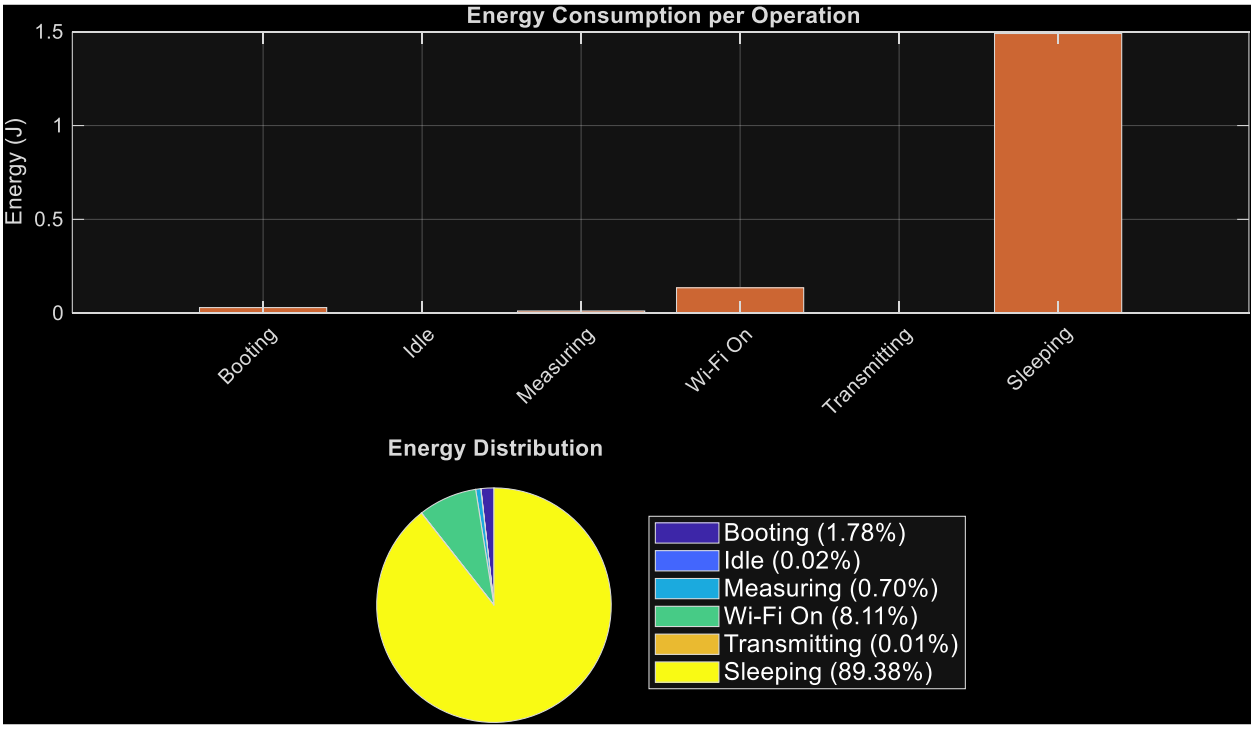
% Q2
energy_values = [boot_energy, idle_energy,measure_energy,wifi_energy,tx_energy,sleep_energy];
durations = [boot_time, idle_time,measure_time,wifi_time,tx_time,sleep_time];
cycle_energy = sum(energy_values); % energy per cycle
cycle_time = sum(durations);
avg_power = cycle_energy / cycle_time * 1000; % Convert to mW
duty_cycle = (cycle_time - sleep_time) / cycle_time * 100;

% Q3
battery_energy = mod(3520,5000) + 15000;
cycle_times = battery_energy/cycle_energy;
total_time = cycle_times * cycle_time / 3600; % hours

fprintf('\nDuty Cycle Analysis:\n');
fprintf('Total cycle time: %.3f seconds\n', cycle_time);
fprintf('Active time: %.3f seconds\n', cycle_time - sleep_time);
fprintf('Duty cycle: %.2f%%\n', duty_cycle);
fprintf('Total energy per cycle: %.3f J\n', cycle_energy);
fprintf('Average power consumption: %.2f mW\n', avg_power);
fprintf('Time the sensor node lasts before changing the battery: %.2f', total_time);
```

Plots and Data Visualization

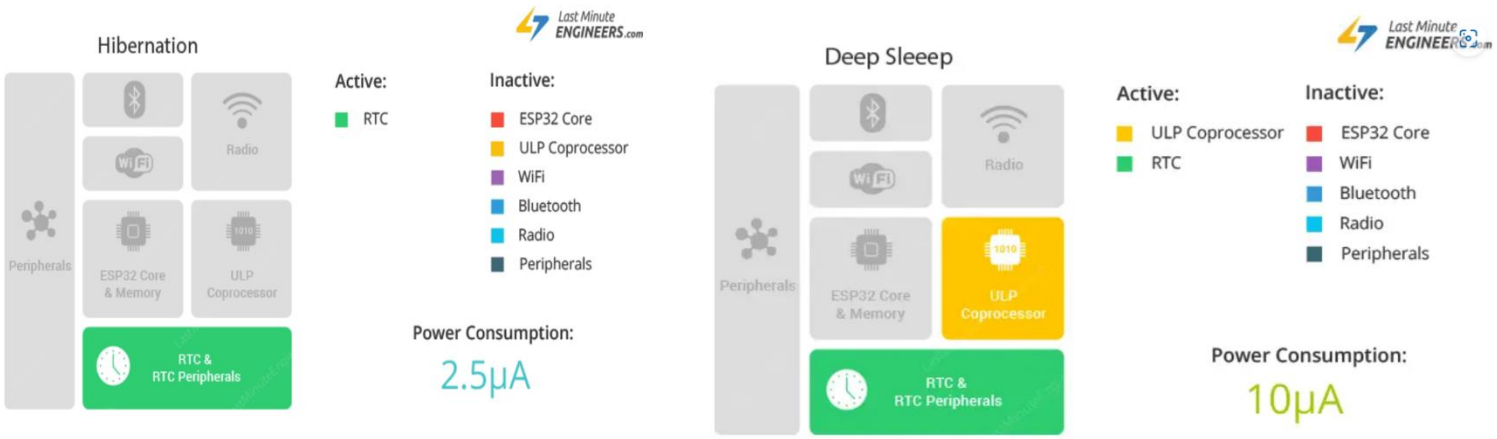




Potential Optimizations [Q3]

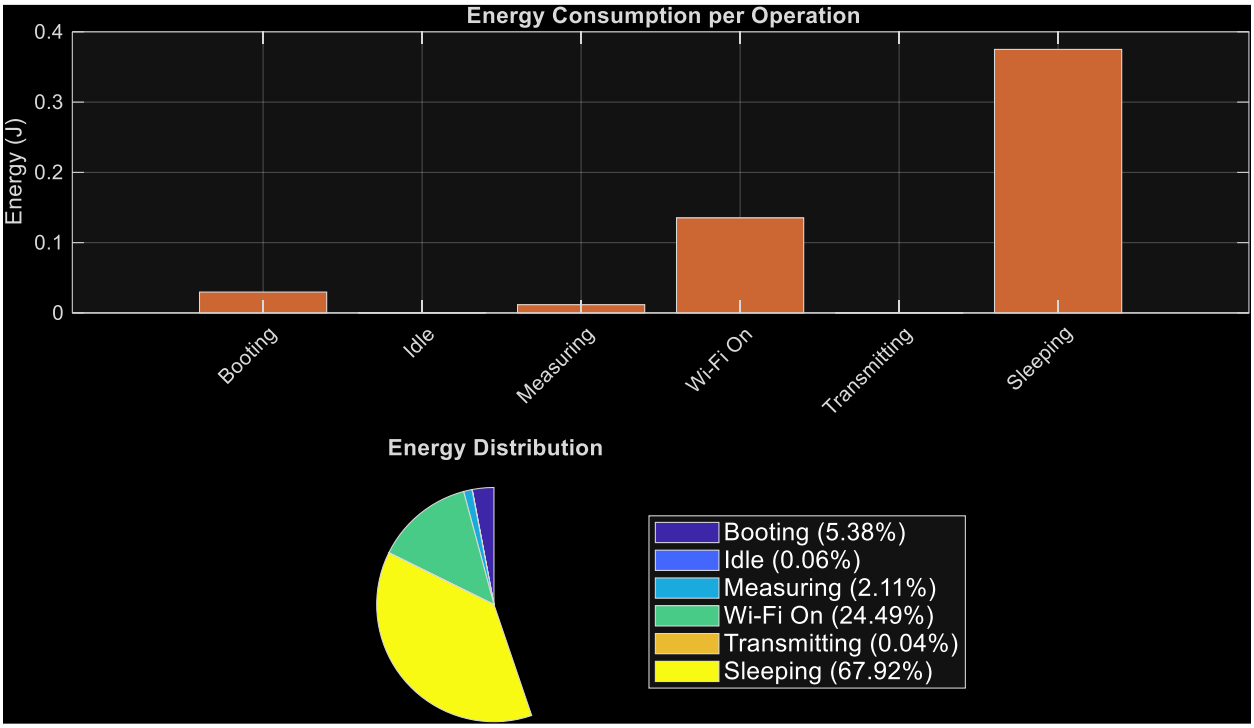
As seen in the visuals, any optimizations we make in the active part will be relatively negligible considering the amount of energy leaking while deep sleeping, the only software-based solution we have is to stop using deep sleep and instead opt for Hibernation Mode.

- Hibernation mode is very similar to deep sleep. The only difference is that in hibernation mode, the chip disables the internal 8 MHz oscillator as well as the ULP-coprocessor, leaving only one RTC timer (on slow clock) and a few RTC GPIOs to wake the chip up.
- Because the RTC recovery memory is also turned off, we cannot save any data while in hibernation mode. This is not an issue for us.



If we use hibernation mode, we can expect deep sleep power draw to fall to at-least a quarter of the current power draw, ie from ~60 to ~15 mW, we can recalculate the energy consumption with this power consumption to get the following:

- Total energy per cycle: 0.552 J (from 1.668 J)
- Average power consumption: 21.83 mW (from 65.92 mW)
- Time the sensor node lasts before changing the battery: 235.71 hours (from 78.04 hrs)



If we focus on the active part of the cycle, assuming Wokwi time measurements are accurate, then I would say my implementation is already highly optimized as it spends essentially no time idling and keeps Wi-Fi and Transmission ON only for as long as they need to be, there is very little that can be done to make Wi-Fi startup faster.

However, after Wi-Fi, the second largest power consumer is booting up, minimizing boot up times can lead to some gains in power draw. From the ESP32 official documentation
[ESP Chip Startup Time Optimization - - — ESP-Techpedia latest documentation]

“The unoptimized startup process usually takes a long time (generally around 300 milliseconds), making it unable to meet the requirements of applications with high real-time requirements (such as turning on/off lights at any time). At the same time, it also accompanies high power consumption, resulting in high average power consumption in Deep-sleep low-power mode. To avoid these problems, fine optimization of the startup process is necessary”.

Applying these software optimizations, we can expect our boot time to drop to somewhere around 30ms, re-running the calculations with the boot-time set at 30ms instead of 80ms, we get:

- Total energy per cycle: 1.649 J (from 1.668 J)
- Average power consumption: 65.32 mW (from 65.92 mW)
- Time the sensor node lasts before changing the battery: 78.76 hours (from 78.04 hours)

So, a noticeable, albeit small, difference.